Deep vision quality control - an example application in the casting industry

Pedro Rocha [1] Nuno Martins [1,2] Fernando Lopes [1,2] Luis A. da Silva Cruz [2,3]

[1]Polytechnic Institute of Coimbra, Coimbra Institute of Engineering, Coimbra, PT

[2]Instituto de Telecomunicações, Coimbra, PT,

[3]Department of Electrical and Computer Engineering, University of Coimbra, Coimbra, PT

a21200219@isec.pt nuno.martins@co.it.pt flopes@isec.pt lcruz@deec.uc.pt

RECPAD 2023

Abstract

In this paper, we explore deep learning techniques to classify manufactured parts as defective or non-defective, by processing grayscale images. We conducted experiments on a publicly available dataset comprising 1300 images of casted submersible pump impellers, utilizing a custom CNN implemented with the Keras framework. We attained a recall rate of 94% for the defective class and a precision of 89% for the non-defective class. Additionally, we discuss qualitative insights gained through the application of Grad-CAM (Gradient-weighted Class Activation Mapping) for better understanding of the model's decision-making process.

Motivation

Effective quality control procedures are essential for manufacturing companies, directly influencing their reputation and the reliability of their products. The most common procedures include geometric and surface inspection, but relying on manual labor for these repetitive tasks introduces variability in quality control outcomes.

In our rapidly evolving industrial landscape, time is an increasingly valuable resource. To meet the demands of 24/7 manufacturing, technology-driven solutions have become indispensable, benefiting both employers and employees. One promising approach is the use of deep learning-based models in computer vision classification tasks, in order to automate the inspection process.

Methodology

Our work focuses on the core of the image processing task for automated inspection in production lines. The implemented system receives grayscale images of submersible pump impellers containing casting defects and outputs a boolean value by applying a sigmoid function to the probability of the presence of defects.

We used a publicly available dataset from Kaggle, provided by Ravirajsinh Dabhi with Pilot Technocast collaboration [1] containing 1300 grayscale 512x512 images. Although the dataset comprises various types of defects, the critical aspect for quality control lies in identifying their absence, making this problem a binary classification with just two classes: 'Defective' and 'OK.'

Some examples of casting defects illustrated in the described dataset are exhibited in Figure 1:

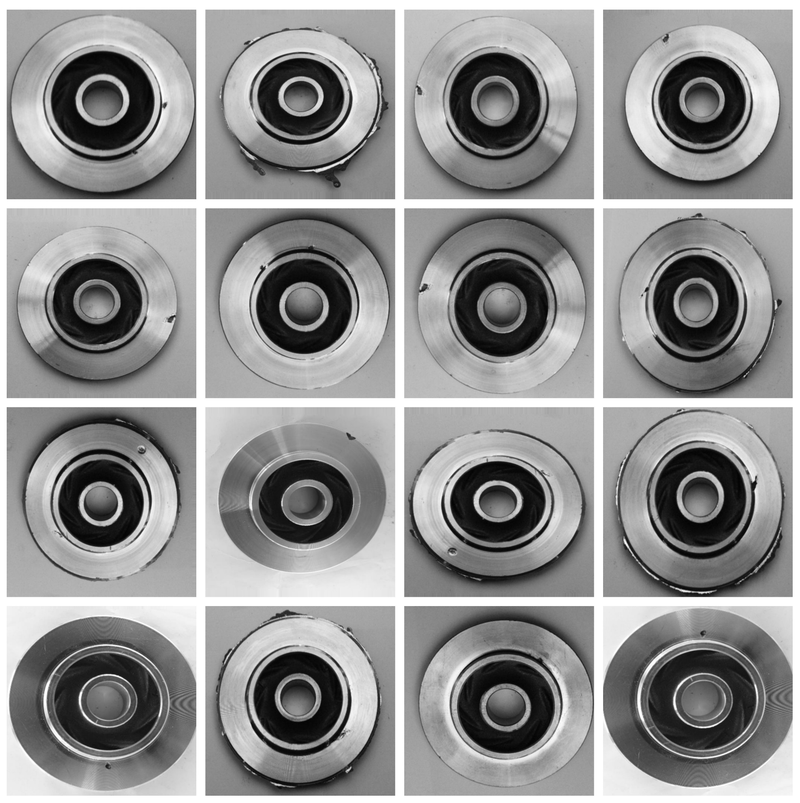


Figure 1. Examples of the diversity of casting defects

Framework

We create a custom Convolutional Neural Network (CNN) using the Keras framework in TensorFlow, defining a sequential model by adding layers according to our designing experience.

Besides adding layers, adjusting layer parameters, and applying regularization techniques, this functional API also enables more complex architectures, allowing to recreate or merge known models from the literature and load the respective trained weights.



Architecture

Our custom model main features are a block of CNN and a block of dense layers, in this specific order:

- $Rescaling(scale = 1.0/255, offset = 0.0)$
- $Conv2D(filters = 32, kernelsize = (5,5), strides = 3, padding =' valid')$
- $Conv2D(filters = 32, kernelsize = (3,3), strides = 2, padding =' valid')$
- $Conv2D(filters = 32, kernelsize = (3,3), strides = 2, padding =' valid')$
- $Conv2D(filters = 64, kernelsize = (3,3), strides = 2, padding =' valid')$
- $Conv2D(filters = 64, kernelsize = (3,3), strides = 2, padding =' valid')$
- $Conv2D(filters = 64, kernelsize = (3,3), strides = 2, padding =' valid')$
- $Flatten()$
- $Dense(64, activation =' relu')$
- $Dense(128, activation =' relu')$
- $Dense(256, activation =' relu')$
- $Dense(1, activation =' sigmoid')$

By starting with an input layer of 512x512x1 we managed to create a computationally efficient network by ending up with 218.881 trainable parameters, with a dimensional structure illustrated in Figure 2.

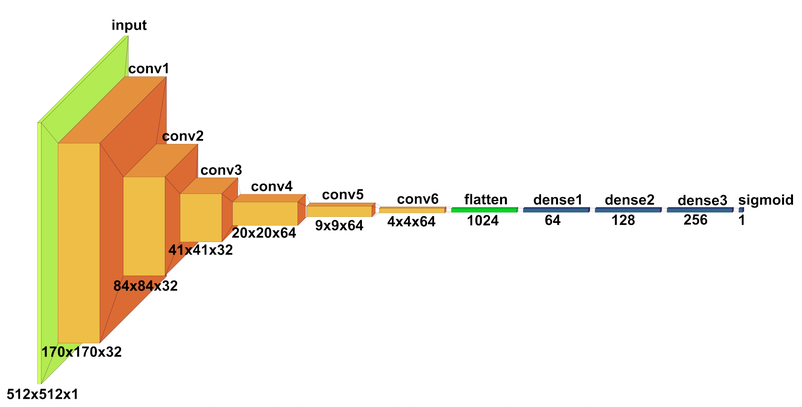


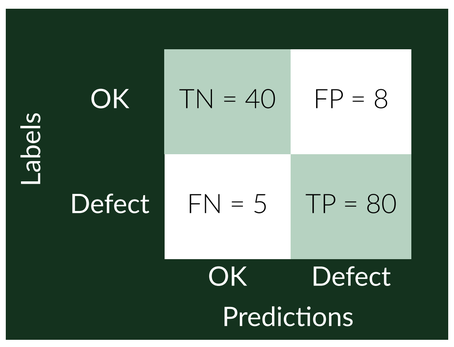
Figure 2. Diagram of the Neural Network

The primary motivation for widening the layers in the dense block was to reduce computational complexity, which could also be achieved by reducing the number of layers. While it may not seem like the most logical direction for such a small network, creating bottlenecks in the model's architecture is often used to increase regularization. Essentially, our choice of these layer parameters was guided by empirical results, even though the potential benefits of this approach may not be immediately apparent.

Results

The confusion matrix presented in Table 1 shows the distribution of the predictions (TN, FN, FN, FP) in the dataset split for testing.

Table 1. Test inference Confusing Matrix (threshold=0.50)



To provide insights into the trade-off between correctly identifying true positives and incorrectly classifying false positives across various classification thresholds, we present the Receiver Operating Characteristic (ROC) curve in Figure 3, where our model achieved an Area Under the ROC Curve (AUC-ROC) of 97%.

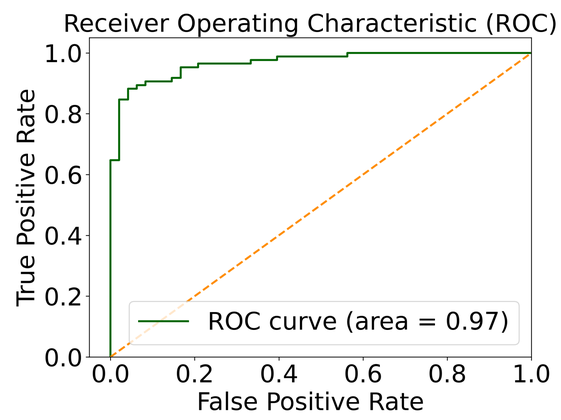


Figure 3. ROC curve

By setting the classification threshold to 0.50, we achieved the values for the metrics detailed in Table 2.

Table 2. Test inference performance metrics (threshold=0.50)

Class	Precision	Recall	F1-Score	# samples
OK	0.89	0.83	0.86	48
Defect	0.91	0.94	0.92	85
Avg / Total	0.90	0.90	0.90	133

Evaluation

Grad-CAM (Gradient-weighted Class Activation Mapping) is a technique used to visualize the regions within an image that most significantly contributed to the final classification decision.

This is achieved by computing the gradients of the decision scores at each pixel in the image with respect to the final classification outcome. These gradients are then weighted and aggregated across a specific convolutional layer's depth.

In Figure 4 and Figure 5, we showcase the results obtained using the Grad-CAM across all six Conv2D layers on images that were correctly classified by our model.

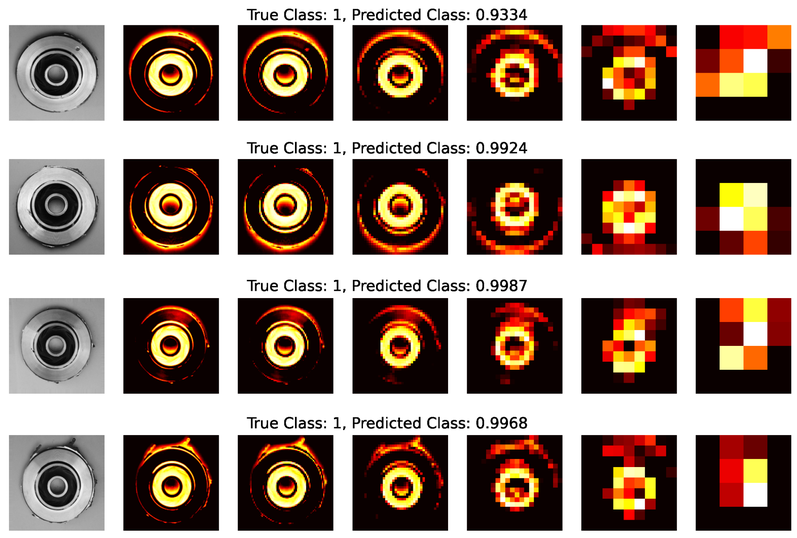


Figure 4. GradCam results of two True Positive examples

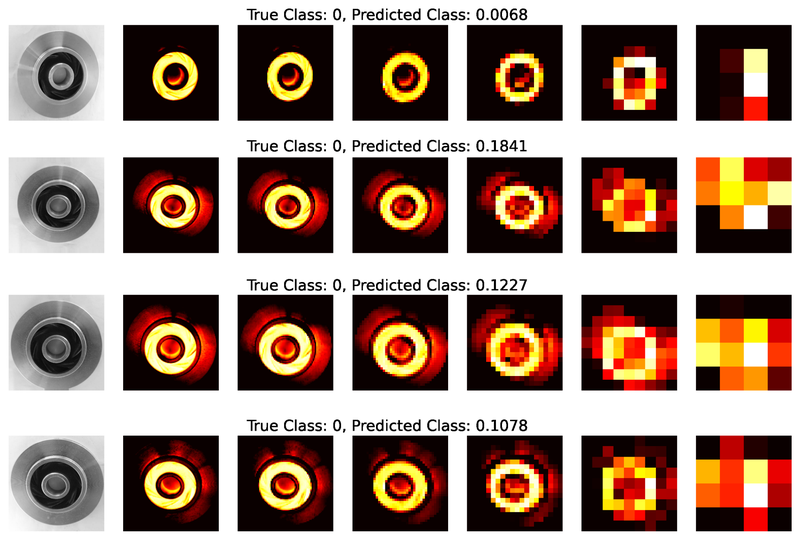


Figure 5. GradCam results of True Negative examples

In Figure 6, we showcase the results obtained on two images that were misclassified by our model.

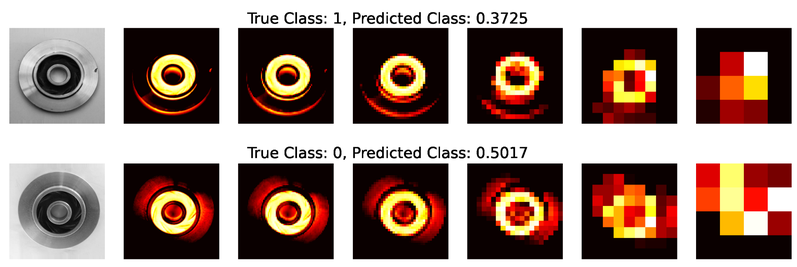


Figure 6. GradCam results of FN and FP examples

In these images, as well as in others, we can observe that the actual defect is indeed highlighted. However, it is essential to note that the defect may not always be the brightest element within the image. Instead, our observation suggests that the model's classification decision is likely influenced by the features of the central ring, which appears to play a significant role in the final decision.

Conclusions and future work

In this study, we assessed the effectiveness of a custom CNN deep learning model in classifying parts from images acquired in a real-world automated inspection line, deciding on their acceptability in terms of quality. The obtained results are good, but still with potential for improvement. In particular, the feature extraction could be enhanced by increasing the quality of the datasets. As part of our future work, we plan to collaborate with a local industry to gain access to a substantial amount of labeled data. In addition, we plan to design an unsupervised learning pre-processing layer to act as a binary mask, bounding the ROI, and making the model decision focus on the actual features rather than noise.

References

[1] Ravirajsinh Dabhi. *Casting Product Image Data for Quality Inspection*. Kaggle, Pilot TechnoCast, Shapar, Rajkot, India, 2020.